Working on Candex

This page orients you if you are going to work on Candex. For the product itself — what it is and how it feels to use — start with Understanding Candex.

The system at a glance

Candex is not one app but a few pieces working together. Users touch the **mobile app**. An **admin dashboard** keeps the drink catalogue accurate. Both talk to a single **REST API**, which holds the domain and stores everything in **PostgreSQL**. Sign-in is handled by **Firebase**.

Repositories

Repository	Role	Status
`candex-app`	Mobile client (Angular + Ionic + Capacitor)	Active
`candex-dashboard`	Admin dashboard for managing the catalogue	Active
`candex-api`	REST API and domain model, backed by PostgreSQL	Active
`candex-handbook`	This documentation site (Astro Starlight)	Active
`candex-crawler`	Crawler that gathers drink and catalogue data	Active
`candex-ai`	AI module (Python)	Early / placeholder
`candex-backups`	Firestore backup tooling	Deprecated — predates the move to PostgreSQL
`candex-api-legacy`	Previous API (Node.js / Express)	Deprecated

> **Note**
>
> `candex-crawler` lives in its own repository (outside the main workspace); its role is summarised here from its purpose and will be detailed in its own handbook.

Tech stack

Mobile app — `candex-app`



- **TypeScript**
- **NgRx** — Store • Effects • Entity
- **RxJS**
- **Firebase** — Auth

Admin dashboard — `candex-dashboard`

- **Angular** — v20
- **Ionic** — v8
- **TypeScript**
- **Angular Material**
- **Firebase** — Auth

Back-end — `candex-api`

- **Spring Boot** — v3.5
- **Java** — 21
- **Maven** — build (wrapper)
- **PostgreSQL**
- **Flyway** — migrations
- **MapStruct** — mapping
- **Spring Security** — OAuth2
- **OpenAPI** — springdoc

Quality & delivery

- **Docker** — Compose
- **JUnit** — + Mockito
- **Testcontainers**
- **JaCoCo** — coverage
- **ESLint**
- **Prettier**
- **Cypress** — e2e

Where things live

- **Domain & API** — `candex-api`. Entities and business rules in `src/main/java`; schema in PostgreSQL, evolved through Flyway migrations under `src/main/resources/db/migration`.
- **Mobile UI** — `candex-app` (Angular + Ionic, NgRx for state).
- **Admin UI** — `candex-dashboard` (Angular + Ionic + Angular Material).

The codebase is the source of truth: when documentation and code disagree, the code wins and the docs are corrected.

Running it locally

Candex runs as three apps — the mobile app and the dashboard (Ionic/Angular, run with npm) and the API (Maven + Spring Boot, with PostgreSQL), each in three environments (local, dev, prod). The full setup — prerequisites, cloning, environment files, the database, and the per-app commands — lives in the **Local Installation** handbook.

Where to go next

- Set up your machine → Local Installation
- Understand how the pieces fit, and why → Architecture & Overview *(planned)*
- The precise domain language → Glossary / Domain *(planned)*

Your first step: clone the repositories, then follow Local Installation to get the API and at least one front-end running against a local database.